

Adversaries and Proof Techniques

2025

1 Introduction

Before it makes any sense to look at any cryptographic construction in detail, we need to consider what kind of dangers we expect to exist in the real world, and how we can argue security against the foreseen threats. Therefore we now consider goals, threats, and way of reasoning about the security against threats (not loosing sight of goals). As a by-product, we (revise) some standard notions used in terms of adversarial powers, and security goals.

1.1 The general setting of multiparty computation (MPC)

Very generally speaking, we consider the setting where multiple parties (or players) $P_1, \dots, P_n$ with inputs $X_1, \dots, X_n$ wish to jointly compute some function $F(X_1, \dots, X_n)$, such that only the function output z is learned by all parties but nothing **else** is learned about the $X_1, \dots, X_n$ (via the messages that are exchanged). This property is sometimes called input privacy.

Notice a caveat here because I highlighted the word else: it is possible that some information about the X_i is leaked to each party simply by learning the value of z . If this sounds strange, then consider the case of two communicating parties who, for instance want to compute the maximum of two values, whereby each party supplies one of the two input values. Obviously, the party who supplies the smaller input, will learn the value that the other party supplied, as this value equates to the function output.

Another property that is typically expected from an MPC protocol is that if an adversary can somehow corrupt a subset of the parties P_i , then they should not be able to get the remaining parties to output an incorrect result.

There are two behaviours for the non-corrupted parties that can be found in MPC protocols. The first one is if the non-corrupted parties are still guaranteed to produce the correct result; this is called robust MPC. The second one is when the non-corrupt parties simply abort the protocol and output nothing; this is called MPC with abort.

Ideas and Concepts Covered in this Write-up.

- Types of adversaries
- Proof philosophy and techniques

2 Types of adversaries

An **adversary** is some entity who wishes to “break” whatever security property we are trying to achieve with or without some help using certain tactics. Thus to precisely characterise adversaries, we need to precisely define the security goal first. Before we can do that, we need to consider though in some generality what kind of “help” they might get and what kind of “tactics” they might deploy.

Very generally speaking, any adversary can be considered to be an (efficient) algorithm. An algorithm has inputs, and it produces outputs. It may interact with other algorithms as well. In the context of cryptography, we often represent an adversary as a “game” that is played in a kind of real-world scenario, in which the adversary (i.e. an algorithm) interacts with some cryptosystem, following some rules (that define the game). If the adversary can win the game, then the cryptosystem is deemed insecure.

Continuing to speak generally, we can consider an adversary who gets basically no help in form of any interaction with the cryptosystem/protocol that they are attacking. Such adversaries can only observe the cryptosystem/protocol as it proceeds, but they cannot interact with it. We call such adversaries passive. In contrast an adversary that can interact with the cryptosystem/protocol, and in the case of MPC perhaps can also “take over” some of the parties, is called active.

In the context of cryptographic protocols that are used to enable multiple parties to compute something securely, an adversary could thus just observe the protocol flow from the outside, but they could also try to “take over” one or multiple parties (such parties are then called corrupt). The first of these two cases is in fact what study in classical

cryptography and the use of “secure” encryption/signature schemes is typically deployed to deal with this problem. The second case is unique to the setting of MPC.

Semi-honest adversary: The corrupted parties co-operate to gather information such as internal states as well as messages that are exchanged during the protocol. The corrupted parties do not deviate from the protocol flow, i.e. they continue to send messages as required by the protocol. The goal is typically to learn the private data of the remaining parties.

Malicious adversary: The corrupted parties co-operate to gather information such as internal states as well as messages. The corrupted parties can deviate from the protocol flow. The goal is typically to subvert the computation of the function.

2.1 Computational assumptions

At the beginning of this section I put the word *efficient* in brackets when referring to an adversary. The word efficient in fact refers to how much “computational power” we allow the adversary to have. I also put the word *break* under inverted commas because also this word needs to be made preciser.

We consider three general notions of security that are different in how “broken” a protocol ends up and how “efficient” and adversary is. These are as follows.

Perfect security. The adversary has unlimited computational powers and yet they cannot break the security notion.

Statistical security. The adversary has unlimited computational powers and they can only break the security notion with an extremely small probability.

Computational security. The adversary has limited computational powers.

2.2 Fundamental limits on corrupted parties

There are some results available that prove very generically that there is a limit on how many parties can be corrupt when consider computationally bounded vs unbounded adversaries. Suppose there are n parties. These fundamental limits are as follows.

Semi-honest security under unbounded adversaries: fewer than $n/2$ corrupt parties can be tolerated.

Semi-honest security under bounded adversaries: up to $n - 1$ corrupt parties can be tolerated.

Malicious security under unbounded adversaries: fewer than $n/3$ corrupt parties can be tolerated.

Malicious security under bounded adversaries: fewer than $n/2$ corrupt parties can be tolerated for a **robust** protocol, and $n - 1$ corrupt parties can be tolerated if we are prepared to accept a non-robust protocol.

3 Simulation based security

We have considered several practical examples for multiparty computation. There is a connection between all these examples: in all cases we want that parties gain “no knowledge” (about some secret inputs) other than what the output of some computation gives.

Formalising this property, and find a way to prove that some protocol has this property (under some well defined adversary, see the previous sections) is all but trivial. After all we wish any such proof to hold for all “sorts of knowledge” that could be “leaked” from a protocol. So is there a way to prove that some protocol gives zero knowledge without requiring to define “knowledge”?

The brilliant idea goes back to two cryptographers, Shafi Goldwasser and Silvio Micali. We will try and gain some understanding of it in this course, and we will do so in multiple stages. In this section, we consider the super high-level idea of the proof technique only.

3.1 Travelling back in time: semantic security

Let us consider just a simple symmetric encryption scheme for the moment. We consider a symmetric encryption scheme as secure if an adversary gains no information about the plaintext when given some ciphertext (sound familiar?). In fact it was Claude Shannon who phrased security in this way, and considered systems that would have this property even if you allow a computationally unbounded adversary; this is the notion of perfect secrecy for symmetric encryption.

Goldwasser and Micali picked this idea up and reframed for computationally bounded adversaries. They did this as follows. Suppose the adversary $\mathcal{A}$ (a polynomially bounded algorithm who gets given a challenge ciphertext, and information about the length of the underlying plaintext) cannot only determine some information about the message

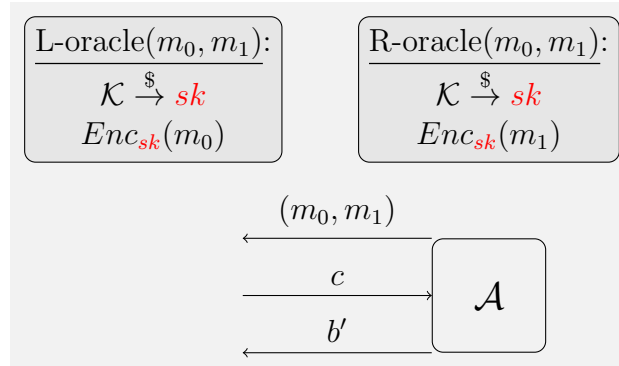


Figure 1: Exp LR-Pas. Win if $b' = b$.

with a probability that is “about the same” as any other adversary $\mathcal{A}'$ who only gets the message length (but not the ciphertext). What this informal definition is saying is that the ciphertext is of no help to the adversary saying something about the message, therefore, it contains no information about the message.

But they struggled initially with the application of this notion and instead suggest and used the (slightly weaker) notion of cipher indistinguishability.

3.1.1 Indistinguishability of ciphertext: left-right security

The idea of indistinguishable ciphertexts is as follows. Imagine that an adversary chooses two plaintexts m_0 and m_1 with the only restriction that they must be of equal length. They then get a ciphertext that either corresponds to the encryption of m_0 (the left oracle) or the encryption of m_1 (the right oracle). If the adversary can successfully decide which oracle was used, then they win the game. If no adversary can win the game for an encryption scheme $\mathcal{E}$ then we call $\mathcal{E}$ perfectly secret.

We can show that this property holds for the one-time pad (OTP).

The proof argument would be as follows. The key generation of the OTP, $\mathcal{K} \xrightarrow{\$} sk$, delivers a key sk with $\Pr[k] = 1/|\mathcal{K}|$. For fixed $m \in \mathcal{M}$, the mapping $c = m \oplus k$ is bijective: i.e. each c is equally likely.

The c returned from the left oracle, and the c returned from the right oracle are therefore both uniformly distributed; aka they cannot be distinguished.

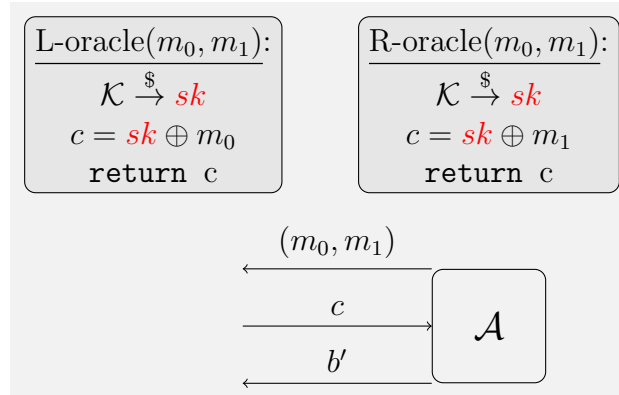


Figure 2: Exp LR-Pas for the OTP. Win if $b' = b$.

3.2 Real vs. ideal world

Now we are going to come back to the simulation based security notion, which is about distinguishing between the “real world” and an “ideal world”, which is characterised by the fact that it has no information about the secret.

There is the adversary $\mathcal{A}$ who gets the ciphertext. This adversary in some sense represents what we expect to happen in the *real world*. There is a ciphertext, which is being sent over some channel, and we wonder if it contains something useful.

There is another adversary $\mathcal{A}'$ as well who actually does not get any ciphertext information. This situation represents in some sense an *ideal situation* in which really no whatsoever information is given to the adversary.

We are then saying that a symmetric encryption scheme is semantically secure if the probability of $\mathcal{A}$ succeeding to learn something is about the same as $\mathcal{A}'$ succeeding in learning something. Why? Well $\mathcal{A}'$ obviously learns nothing as they have nothing given to them that relates to the message, therefore, also $\mathcal{A}$ learns nothing.

So the mental model of the proof is as follows: define the real world (by defining an adversary and what they actually get); then define how this would look like in the ideal world; finally show that in some sense these worlds are “equivalent”.

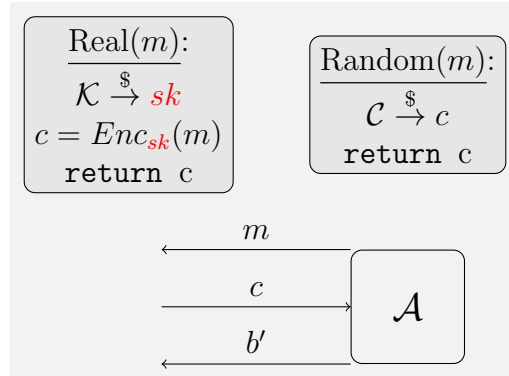


Figure 3: Exp RR-Pas. Win if $b' = b$.

3.3 Simulation

The need to show that the real and ideal world are the same (or demonstrate a problem by showing that they are not) requires another idea. In the paper by Goldwasser and Micali, they actually then “circumvent” this problem by showing “semantic security” is equivalent to “ciphertext indistinguishability” (which I explained before).

The solution to the challenge of finding a general technique to show equivalence between the real and the ideal world is based on defining a simulator that is capable of simulating the ideal world in such a way that an adversary cannot tell whether they are given “stuff” that comes from a protocol run in the real world, or whether they are given “stuff” that comes from a protocol run in the ideal world.

Notice that by virtue of this principle, we don’t ever have to define what kind of “knowledge” they might want to extract. The ideal world, because it is set up so that it does not relate to any secret at all, cannot give any information (no matter how we define information or knowledge). Therefore if the real protocol flow is indistinguishable from the protocol flow in the ideal world, then the real protocol cannot contain any information either about the secret.

3.4 Toy example

It is not straightforward to illustrate the principle of simulation based proofs without getting immediately rather technical. This example is my “best effort” to come up with something meaningful, yet easy to play with.

Suppose that we consider protocol with three parties P_1, P_2, P_3 . The three parties each hold a value x_i such that x_1 and x_2 were randomly chosen from a uniform distribution of size n , and $x_3 = x \oplus x_1 \oplus x_2$. The value x is not known to any of the parties. We can see from the construction so far that x_1 and x_2 are independent of x simply because they were selected at random. The value x_3 is also independent of x because it can be understood as a “one-time” pad encryption of x with the one-time secret $x_1 \oplus x_2$. Now suppose that P_1 and P_2 compute $x_1 \wedge x_2$. The protocol may be repeatedly executed, whereby in each protocol run, x_1 and x_2 are newly sampled (but x remains the same).

Assume at first that an adversary only gets information on shares x_i . Suppose that a single P_i is corrupted: any x_1 and x_2 are drawn from a uniform distribution, thus the simulator for the random world just samples a value from the uniform distribution. If x_3 is leaked, then observe that $x_3 = s \oplus x_1 \oplus x_2$, thus x_3 is the one-time pad encryption of s . We know that this cannot be distinguished from drawing x_3 uniformly, hence the simulator can produce x_3 in the same way as it produces x_1 and x_2 .

Now suppose that there an adversary can also see the result of $x_1 \cdot x_2$. If P_1 **or** P_2 **corrupted**, then the simulator has a bit more to do. This is because if $x_1 \cdot x_2$ is, e.g. 0 and $x_1 = 1$, then $\Pr[x_2 = 0] = 1$. Thus now the simulator must actually know=choose x_2 in order to produce a perfect simulation (i.e. ensure that the distribution of values in the random world cannot be distinguished from the distribution of values from the real world).